فصل ۳: CSS

> **پیش‌نیاز:** فصل ۲ (HTML). بدون ساختار درست، CSS فقط ظاهر یک ساختار خراب را «آرایش» می‌کند. **این فصل سه سطح دارد:** مقدماتی، متوسط، پیشرفته. در پایان هر سطح تمرین، «آیا بلدم؟» و پروژه آمده و پایان فصل چک‌لیست مهارتی CSS است.
>
> **نکته‌ی ویژه‌ی فارسی:** تمام مثال‌ها با `dir="rtl"` روی `html` در نظر گرفته شده‌اند. هر جا رفتار CSS در حالت راست‌به‌چپ فرق می‌کند، با برچسب «RTL» مشخص کرده‌ام.

نقشه‌ی فصل

بخش	موضوع	اهمیت
۳.۱	CSS چیست و چطور اضافه می‌شود	[ضروری]
۳.۲	Selectorها	[ضروری]
۳.۳	رنگ‌ها	[ضروری]
۳.۴	واحدها	[ضروری]
۳.۵	Box Model	[ضروری]
۳.۶	Background	[ضروری]
۳.۷	فونت و متن	[ضروری]
۳.۸	display	[ضروری]
۳.۹ تا ۳.۱۳	CSS متوسط: position، Flexbox، Grid، Responsive، انیمیشن	[ضروری]
۳.۱۴ تا ۳.۲۵	CSS پیشرفته: متغیرها، Container، Stacking، Specificity، Query، معماری، دسترس‌پذیری، UI واقعی	[ضروری] تا [پیشرفته/اختیاری]

بخش A: CSS مقدماتی

۳.۱ CSS چیست؟ [ضروری]

مفهوم. CSS (Cascading Style Sheets) زبانی است که **ظاهر و چیدمان** المان‌های HTML را تعیین می‌کند. HTML می‌گوید «این یک عنوان است»؛ CSS می‌گوید «آبی، بزرگ و وسط‌چین باشد».

ساختار یک قاعده (Rule):

```
h1 {
  color: #1d4ed8;
  font-size: 2rem;
}
```

```
h1            { color : #1d4ed8 ; }
 |              |        |
 selector       property value
                \_________________/
                    declaration
```

- **Selector:** کدام المان‌ها را انتخاب کن.
- **Declaration:** هر جفت `property: value;` .
- **Declaration Block:** همه‌ی declarationها بین `{ }` .

سه روش اضافه کردن CSS:

```
<!-- ۱) Inline: مستقیم روی المان (اولویت بالا، نگهداری سخت: پرهیز کن) -->
<p style="color: red;">متن</p>

<!-- ۲) Internal: داخل style در head (مناسب نمونه‌ی کوچک) -->
<style>
  p { color: red; }
</style>

<!-- ۳) External: فایل جدا (روش اصلی پروژه‌ها) -->
<link rel="stylesheet" href="css/style.css" />
```

روش	مزیت	عیب
Inline	سریع برای تست	بازاستفاده ندارد، Specificity بالا، شلوغی HTML
Internal	یک فایل	فقط برای همان صفحه
External	بازاستفاده، کش مرورگر، تفکیک وظایف	یک درخواست اضافه

کامنت در CSS: `/* توضیح */` (فقط همین شکل؛ `//` در CSS کامنت نیست).

اشتباهات رایج

- فراموش کردن `;` بین declarationها (declaration بعدی ممکن است نادیده گرفته شود).
- اشتباه در مسیر فایل CSS. اگر استایل اعمال نشد، اول تب Network و Console را نگاه کن.
- خطای املایی در property: CSS خطا نمی‌دهد، فقط آن خط نادیده گرفته می‌شود. در DevTools خط خط‌خورده می‌بینی.

> **تمرین کوچک:** `style.css` را به صفحه‌ی ساده‌ات لینک کن، رنگ `h1` را عوض کن و بعد یک اشتباه املایی عمدی در property بنویس و در DevTools ببین چه می‌شود.

۳.۲ Selectorها [ضروری]

Selectorهای پایه

```
/* المان */
p { line-height: 1.8; }

/* کلاس (با نقطه) */
.card { padding: 16px; }

/* شناسه (با #) */
#pricing { margin-top: 40px; }

/* همه‌ی المان‌ها */
* { box-sizing: border-box; }

/* گروه‌بندی: یک قانون برای چند selector */
h1, h2, h3 { font-weight: 700; }

/* ترکیب: المان p که کلاس note دارد (بدون فاصله) */
p.note { color: #92400e; }
```

Selectorهای ترکیبی (Combinators)

```
/* نسل‌ها: هر li داخل nav (در هر عمقی) */
nav li { display: inline-block; }

/* فرزند مستقیم */
ul > li { list-style: none; }

/* هم‌سطح بعدیِ بلافاصله */
h2 + p { font-size: 1.1rem; }

/* همه‌ی هم‌سطح‌های بعدی */
h2 ~ p { color: #475569; }
```

Combinator	نماد	معنی
Descendant	فاصله	هر نسلی داخل
Child	`>`	فرزند مستقیم
Adjacent sibling	`+`	برادر بلافاصله بعدی
General sibling	`~`	همه‌ی برادرهای بعدی

Attribute Selector

```
 a[target="_blank"] { color: #7c3aed; }
input[type="email"] { direction: ltr; }
a[href^="https"] { }        /* شروع شود https با */
a[href$=".pdf"] { }         /* تمام شود pdf. با */
a[href*="example"] { }      /* شامل example باشد */
```

Pseudo-class: وضعیت المان (با یک `:`)

```
 a:hover { text-decoration: underline; }        /* ماوس روی آن */
button:focus-visible { outline: 3px solid #2563eb; } /* فوکوس با کیبورد */
input:checked + label { font-weight: 700; }    /* انتخاب‌شده checkbox */
li:first-child { font-weight: 700; }
li:last-child { border: 0; }
li:nth-child(2n) { background: #f1f5f9; }      /* زوج‌ها */
li:not(.active) { opacity: .7; }
input:invalid { border-color: #dc2626; }
```

Pseudo-element: بخشی از المان (با دو `::`)

```
 p::first-line { font-weight: 700; }
input::placeholder { color: #94a3b8; }
::selection { background: #fde68a; }

.badge::before {
  content: "جدید";    /* بدون content کار نمی‌کند */
  background: #dc2626;
  color: #fff;
  padding: 2px 6px;
  border-radius: 4px;
}
```

فرق Pseudo-class و Pseudo-element:

	Pseudo-class	Pseudo-element
نماد	`:`	`::`
یعنی	المان در **وضعیت** خاص	**بخشی** از المان یا بخش ساختگی
مثال	`:hover` ، `:first-child`	`::before` ، `::after` ، `::placeholder`

نکته: محتوای تولیدشده با `::before` و `::after` در DOM وجود ندارد. محتوای مهم را (که باید خوانده یا انتخاب شود) در آن‌ها نگذار.

اشتباهات رایج

- فراموش کردن `.` برای class یا `#` برای id.
- `ul li` را با `ul > li` یکی گرفتن (اولی همه‌ی نسل‌ها).
- استفاده از `id` برای استایل‌دهی. (Specificity بالا؛ در بخش پیشرفته می‌بینی.)
- نوشتن `::before` بدون `content` .
- Selectorهای خیلی طولانی مثل `div.container ul li a span` ؛ شکننده‌اند.

> **تمرین کوچک:** در یک لیست ۶تایی، ردیف‌های زوج را رنگ دیگری بده، اولین آیتم را ضخیم کن و بعد از متن آخرین آیتم یک ✓ (با `::after`) اضافه کن.

۳.۳ رنگ‌ها [ضروری]

```
.demo {
  color: tomato;                    /* نام رنگ */
  color: #1d4ed8;                   /* رقم Hex: ۶ */
  color: #1d4ed8cc;                 /* Hex با شفافیت (۸ رقم) */
  color: rgb(29, 78, 216);          /* RGB */
  color: rgb(29 78 216 / 0.8);      /* مدرن RGB با شفافیت */
  color: hsl(224, 76%, 48%);        /* Hue, Saturation, Lightness */
  background-color: transparent;
}
```

فرمت	چه زمانی؟
Hex	رایج‌ترین، از طراحی (Figma) کپی می‌شود
`rgb()`	وقتی شفافیت یا محاسبه لازم است
`hsl()`	وقتی می‌خواهی تیره‌تر یا روشن‌تر کنی (فقط Lightness را عوض کن). برای ساخت پالت رنگ عالی است

کنتراست: متن باید روی پس‌زمینه کنتراست کافی داشته باشد (راهنمای WCAG: حداقل ۴.۵ به ۱ برای متن معمولی). خاکستری کم‌رنگ روی سفید خیلی رایج و خیلی بد است. DevTools مقدار کنتراست را نشان می‌دهد.

اشتباهات رایج: انتخاب رنگ فقط بر اساس سلیقه بدون سنجش کنتراست؛ استفاده از رنگ تنها برای انتقال معنا (مثلاً خطا فقط قرمز).

> **تمرین کوچک:** یک رنگ اصلی را با `hsl` بنویس و سه سایه‌ی روشن‌تر و تیره‌تر از آن بساز (فقط عدد Lightness را عوض کن).

۳.۴ واحدها [ضروری]

واحد	نسبت به	مثال و کاربرد
`px`	پیکسل ثابت	border، سایه، جزئیات کوچک
`%`	**والد** (برای width)	`width: 50%`
`rem`	اندازه‌ی فونت **ریشه** (`html`، معمولاً ۱۶px)	فونت و فاصله‌ها (مقیاس‌پذیر با تنظیمات کاربر)
`em`	اندازه‌ی فونت **همین المان** (برای font-size: والد)	فاصله‌هایی که باید با فونت همان المان مقیاس شوند (مثل padding دکمه)
`vw` / `vh`	۱٪ عرض / ارتفاع **viewport**	بخش تمام‌صفحه
`ch`	عرض کاراکتر `0`	محدود کردن عرض خط متن (`max-width: 65ch`)

```
 html { font-size: 100%; }          /* همان ۱۶px، اما با احترام به تنظیم کاربر */
h1 { font-size: 2.5rem; }           /* ۴۰px */
p { max-width: 65ch; }              /* خط خوانا */
.hero { min-height: 100vh; }
button { padding: .75em 1.5em; }    /* با اندازه‌ی فونت دکمه مقیاس می‌شود */
```

قاعده‌ی عملی:

- **فونت:** `rem` (نه `px`)، تا کاربری که فونت مرورگر را بزرگ کرده، اثرش را ببیند.
- **فاصله‌ها:** `rem` یا `em`.
- **جزئیات بسیار ریز** (خط ۱px، سایه): `px`.
- **عرض‌های انعطاف‌پذیر:** `max-width`، `fr`، `%`.

> **نکته‌ی موبایل:** `100vh` روی موبایل ممکن است با نوار مرورگر هماهنگ نباشد؛ واحدهای `dvh` و

`svh` راه مدرن‌اند (پشتیبانی را در MDN بررسی کن).

اشتباهات رایج

- `em` تو در تو که اندازه‌ها را ناخواسته چند برابر می‌کند (مثلاً `em` روی لیستِ داخل لیست).
- `px` برای فونت‌های بدنه.
- `width: 100vw` که به خاطر اسکرول‌بار، افقی اسکرول می‌سازد (از `100%` استفاده کن).

تمرین کوچک: `html` را `font-size: 100%` بگذار و همه‌ی اندازه‌های یک صفحه را به `rem` ببر. بعد در تنظیمات مرورگر اندازه‌ی فونت را بزرگ کن و ببین صفحه چطور پاسخ می‌دهد.

۳.۵ Box Model [ضروری]

هر المان یک **جعبه** است با چهار لایه، از داخل به بیرون:

```
 +----------------------------- margin -------------------------------+
|  +------------------------- border ----------------------------+  |
|  |  +---------------------- padding ------------------------+  |  |
|  |  |                                                       |  |  |
|  |  |                  content (width x height)             |  |  |
|  |  |                                                       |  |  |
|  |  +-------------------------------------------------------+  |  |
|  +-------------------------------------------------------------+  |
+-----------------------------------------------------------------+
```

لایه	وظیفه
content	خود محتوا (با `width` و `height`)
padding	فاصله‌ی **داخلی**، بین محتوا و border (پس‌زمینه را شامل می‌شود)
border	حاشیه
margin	فاصله‌ی **بیرونی** از المان‌های دیگر (شفاف)

```
.card {
  width: 300px;
  padding: 16px;                    /* ۴ طرف */
  border: 2px solid #cbd5e1;
  margin: 24px auto;                /* بالا/پایین ۲۴، چپ/راست خودکار (وسط‌چین) */
  border-radius: 12px;
}
```

ترتیب Shorthand:

```
 margin: 10px;                  /* همه */
margin: 10px 20px;              /* بالا و پایین | چپ و راست */
margin: 10px 20px 30px;         /* بالا | چپ و راست | پایین */
margin: 10px 20px 30px 40px;    /* بالا | راست | پایین | چپ (ساعت‌گرد) */
```

`box-sizing`: مهم‌ترین تنظیم

به‌طور پیش‌فرض (`content-box`) فقط **محتوا** (`width`) را تعیین می‌کند و padding و border اضافه می‌شوند. این یعنی `width: 300px` با `padding: 20px` و `border: 2px` در عمل ۳۴۴px جا می‌گیرد، که چیدمان را به‌هم می‌ریزد.

```
/* مدرن و رایج: width شامل padding و border هم می‌شود */
*, *::before, *::after {
  box-sizing: border-box;
}
```

Margin Collapse (ادغام margin): margin عمودی دو بلوک مجاور **ادغام می‌شود** و بزرگ‌ترین مقدار اثر می‌کند (مثلاً ۲۰ و ۳۰ می‌شود ۳۰، نه ۵۰). این در Flex و Grid اتفاق نمی‌افتد.

وسط‌چین افقی بلوک: `margin-inline: auto` (یا `margin: 0 auto`) با `width` یا `max-width` مشخص.

اشتباهات رایج

- فراموش کردن `box-sizing: border-box`.
- اشتباه گرفتن padding و margin (پس‌زمینه در padding دیده می‌شود نه margin).
- `margin: auto` روی المان inline (فقط روی بلوک کار می‌کند).
- `height` ثابت برای متن (وقتی متن بیشتر شود سرریز می‌کند؛ بهتر است `min-height` بگذاری).

> **تمرین کوچک:** دو `div` پشت هم بساز با `margin-bottom: 20px` و `margin-top: 30px`. فاصله‌ی واقعی بین آن‌ها را در DevTools (نمایش Box Model) بخوان.
>
> **RTL و ویژگی‌های منطقی (Logical Properties):** به‌جای `margin-left` و `margin-right` که به جهت فیزیکی وابسته‌اند، می‌توانی بنویسی `margin-inline-start` (ابتدای خط) و `margin-inline-end` (انتهای خط). در `dir="rtl"` ابتدا راست است. در بخش پیشرفته بیشتر می‌بینی.

۳.۶ Background [ضروری]

```
.hero {
  background-color: #0f172a;
  background-image: url("../images/hero.jpg");
  background-size: cover;          /* کل کادر را پر کن (ممکن است برش بخورد) */
  background-position: center;
  background-repeat: no-repeat;
}

/* گرادیان (به‌عنوان تصویر) */
.banner {
  background: linear-gradient(135deg, #1d4ed8, #7c3aed);
}

/* چند پس‌زمینه */
.hero {
  background:
    linear-gradient(rgb(0 0 0 / .5), rgb(0 0 0 / .5)),
    url("../images/hero.jpg") center / cover no-repeat;
}
```

خاصیت	مقادیر رایج
`background-size`	`cover` (پر می‌کند و می‌برد)، `contain` (کامل نشان می‌دهد)
`background-position`	`center` ، `top` ، `20% 50%`
`background-repeat`	`no-repeat` ، `repeat-x`

چه موقع `<img>` و چه موقع `background-image`؟ تصویری که **بخشی از محتوا** است (عکس محصول، نمودار) باید `<img>` با `alt` باشد. تصویر **تزئینی** با `background-image`. (پس‌زمینه‌ی CSS به screen reader چیزی نمی‌گوید.)

نکته‌ی مرتبط: برای کنترل اندازه‌ی `<img>` در کادر: `object-fit: cover` (مثل `background-size: cover`)؛ این‌ها را در بخش متوسط ببین.

اشتباه رایج: گذاشتن متن مهم روی تصویر بدون پوشش تیره یا گرادیان؛ خوانایی خراب می‌شود.

۳.۷ فونت و متن [ضروری]

```
 body {
  font-family: "Vazirmatn", Tahoma, "Segoe UI", sans-serif;
  font-size: 1rem;
  font-weight: 400;
  line-height: 1.8;
  color: #1e293b;
}

h1 {
  font-size: 2.25rem;
  font-weight: 800;
  line-height: 1.3;
  text-align: center;
  letter-spacing: -0.01em;
}

a { text-decoration: none; }
.title { text-transform: uppercase; }
```

خاصیت	توضیح
`font-family`	**لیست اولویت** (Font Stack)؛ آخرین مورد یک خانواده‌ی عمومی (`sans-serif` ، `serif` ، `monospace`)
`font-size`	اندازه (`rem`)
`font-weight`	ضخامت: ۴۰۰ معمولی، ۷۰۰ ضخیم
`line-height`	فاصله‌ی خطوط. **بدون واحد** بنویس (`1.8`)؛ برای فارسی ۱.۷ تا ۲ معمولاً مناسب است
`text-align`	`justify` ، `center` ، `left` ، `right` و منطقی: `start` ، `end`
`text-decoration`	زیرخط (`none` ، `underline`)
`text-transform`	تغییر حروف انگلیسی (`capitalize` ، `uppercase`). روی فارسی اثری ندارد
`letter-spacing`	فاصله‌ی حروف. **در فارسی و عربی نباید بزرگ شود**، چون حروف به هم متصل‌اند و اتصال شکسته می‌شود

استفاده از فونت وب (مثلاً Vazirmatn)

فونت باید در سیستم کاربر باشد یا خودت آن را بارگذاری کنی:

```
@font-face {
  font-family: "Vazirmatn";
  src: url("../fonts/Vazirmatn-Regular.woff2") format("woff2");
  font-weight: 400;
  font-style: normal;
  font-display: swap;    /* ابتدا فونت جایگزین را نشان بده */
}

@font-face {
  font-family: "Vazirmatn";
  src: url("../fonts/Vazirmatn-Bold.woff2") format("woff2");
  font-weight: 700;
  font-display: swap;
}
```

- فایل فونت را در پروژه (`fonts/`) نگه می‌داری و با `@font-face` معرفی می‌کنی. فرمت `woff2` سبک‌تر و برای وب مناسب است.
- `font-display: swap` از «متن نامرئی» تا لود فونت جلوگیری می‌کند.
- می‌توانی فونت را از CDN هم بگیری؛ ولی فایل محلی کنترل و حریم خصوصی بهتری می‌دهد.

اشتباهات رایج

- `line-height` با واحد `px` (در اندازه‌های مختلف فونت به‌هم می‌خورد).
- استفاده از فونت بدون fallback.
- لود چند وزن فونت غیرضروری (هر وزن یک فایل است).
- `letter-spacing` مثبت روی متن فارسی.
- استفاده از `text-align: justify` روی متن‌های کوتاه یا ستون‌های باریک (فاصله‌های ناجور).

> **تمرین کوچک:** یک متن فارسی بلند را با سه مقدار `line-height` (۱.۳ و ۱.۸ و ۲.۴) ببین و عدد مناسب را انتخاب کن؛ بعد `max-width: 65ch` بده و ببین خوانایی چقدر بهتر می‌شود.

۳.۸ `display` [ضروری]

`display` نوع جعبه‌ی المان را تعیین می‌کند.

مقدار	رفتار
`block`	کل عرض، خط جدید؛ `margin` ، `height` ، `width` کامل کار می‌کنند
`inline`	در ادامه‌ی متن؛ `width` / `height` و margin عمودی اثر ندارد
`inline-block`	در ادامه‌ی خط، ولی `height` ، `width` و padding/margin کامل کار می‌کنند
`none`	المان کاملاً حذف می‌شود (جا نمی‌گیرد و برای screen reader هم نیست)
`flex` / `grid`	چیدمان مدرن (بخش بعد)

```
nav a { display: inline-block; padding: 8px 12px; }
.hidden { display: none; }
```

روش‌های مخفی کردن؛ تفاوت‌ها:

روش	جا می‌گیرد؟	دیده می‌شود؟	screen reader؟
`display: none`	خیر	خیر	خیر
`visibility: hidden`	بله	خیر	خیر
`opacity: 0`	بله	شفاف (ولی هنوز قابل‌کلیک و فوکوس)	بله

اشتباهات رایج

- دادن `width` به المان `a` ، `span`) `inline`) و تعجب از بی‌اثر بودنش.
- استفاده از `opacity: 0` به‌عنوان «مخفی» (المان هنوز فوکوس می‌گیرد).
- `display: inline-block` و فاصله‌ی ناخواسته‌ی بین المان‌ها (به‌خاطر فاصله‌ی HTML). با Flexbox حل می‌شود.

تمرین‌های CSS مقدماتی

ساده

1. در صفحه‌ی HTML فصل قبل، فونت، رنگ متن و `line-height` را تنظیم کن.
2. به همه‌ی المان‌ها `box-sizing: border-box` بده و با DevTools تفاوت را نشان بده.

3. لینک‌ها را بدون زیرخط کن و با `:hover` زیرخط بده.

متوسط

1. یک «کارت» بساز: عرض ثابت، border، padding، گوشه‌ی گرد، وسط‌چین با `margin: auto`.
2. با `::before` یک نشان «جدید» به کارت اضافه کن.
3. دو پس‌زمینه‌ی گرادیان متفاوت برای دو بخش صفحه بساز.

چالشی

1. بدون فریم‌ورک، یک «نوار ناوبری» افقی با hover، `inline-block` و وضعیت active بساز (بعد از یادگیری Flexbox همین را دوباره بساز و تفاوت را ببین).

آیا CSS مقدماتی را بلدم؟

سؤالات مفهومی

1. چرا `box-sizing: border-box` تقریباً همیشه تنظیم می‌شود؟
2. فرق `rem` و `em` چیست و برای فونت بدنه کدام بهتر است؟
3. چرا `a:hover` یک Pseudo-class و `a::after` یک Pseudo-element است؟
4. چرا نباید `id` را برای استایل‌دهی زیاد به‌کار برد؟
5. چرا `line-height: 1.8` از `line-height: 28px` بهتر است؟
6. فرق `display: none` و `visibility: hidden` و `opacity: 0` چیست؟

تمرین کوتاه (کد را بخوان)

1. عرض واقعی این المان روی صفحه چقدر است؟ و اگر `box-sizing: border-box` بود چقدر می‌شد؟

```
.box {
  width: 200px;
  padding: 20px;
  border: 5px solid black;
}
```

1. کدام المان‌ها با این selectorها انتخاب می‌شوند؟

```
ul li { }
ul > li { }
h2 + p { }
h2 ~ p { }
```

برای این HTML:

```
<ul>
  <li>A
    <ul><li>B</li></ul>
  </li>
</ul>
<h2>عنوان</h2>
<p>۱</p>
<p>۲</p>
```

1. چرا این استایل کاری نمی‌کند؟ (دو دلیل)

```
span { width: 200px; margin-top: 30px; }
.badge::before { color: red; }
```

پروژه‌ی سطح CSS مقدماتی: «استایل‌دهی صفحه‌ی معرفی شخصی»

- **هدف:** استایل‌دهی پروژه‌ی «صفحه‌ی معرفی شخصی» فصل ۲ و ساخت یک ظاهر یکپارچه.
- **امکانات:** فونت و رنگ‌های هماهنگ، ناوبری افقی (با Hero)، `inline-block` با پس‌زمینه‌ی گرادیان یا تصویر، کارت‌های علاقه‌مندی‌ها، حالت hover و focus برای لینک‌ها، یک فوتر.
- **مهارت‌ها:** Selector، رنگ، واحد، Box Model، Background، فونت، `display`.
- **محدودیت‌ها:** فقط یک فایل CSS خارجی؛ بدون Flexbox/Grid؛ بدون inline style؛ همه‌ی اندازه‌های فونت و فاصله با `rem`؛ بدون فریم‌ورک.
- **معیار قبولی:**
- نسبت کنتراست همه‌ی متن‌ها حداقل ۴.۵ به ۱ (با DevTools بسنج).
- `box-sizing: border-box` سراسری.
- حالت `:hover` و `:focus-visible` برای لینک‌ها مشخص و واضح.
- هیچ `id` برای استایل استفاده نشده است.
- صفحه را با ۲۰۰٪ زوم مرورگر بخوان و چیزی از هم نپاشد.

بخش B: CSS متوسط

۳.۹ Position، z-index و overflow [ضروری]

`position` تعیین می‌کند المان چطور جایگذاری شود. با `left` ، `bottom` ، `right` ، `top` (یا مختصر `inset`) جابه‌جا می‌شود.

مقدار	رفتار	نسبت به
`static`	پیش‌فرض؛ در جریان عادی؛ `top/left` اثر ندارد	
`relative`	در جریان عادی می‌ماند و جای اصلی‌اش محفوظ است؛ می‌توان با `top/left` جابه‌جایش کرد	جای اصلی خودش
`absolute`	از جریان عادی **خارج** می‌شود و جایش خالی می‌ماند	نزدیک‌ترین **والد غیر static** (یا کل صفحه)
`fixed`	از جریان خارج و ثابت در صفحه (حتی با اسکرول)	**viewport**
`sticky`	تا یک آستانه‌ی اسکرول مثل `relative` ، بعد مثل `fixed` می‌چسبد	والد اسکرول‌دار

الگوی طلایی: والد `relative` + فرزند `absolute`

```
 .card { position: relative; }

.card .badge {
  position: absolute;
  top: 12px;
  inset-inline-end: 12px;    /* سمت راست در LTR؛ سمت چپ در RTL */
}
```

```
 <div class="card">
  <span class="badge">تخفیف</span>
  <h3>دوره‌ی CSS</h3>
</div>
```

هدر چسبان:

```
.site-header {
  position: sticky;
  top: 0;            /* بدون sticky، top عمل نمی‌کند */
  background: #fff;
  z-index: 100;
}
```

`z-index`: ترتیب لایه‌ها روی محور عمق. فقط روی المان‌هایی اثر دارد که `position` غیر `static` (یا فرزند Flex/Grid) باشند. عدد بزرگ‌تر بالاتر است. (نکته‌ی مهم درباره‌ی Stacking Context در بخش پیشرفته.)

`overflow`: رفتار وقتی محتوا از جعبه بیرون می‌زند.

مقدار	رفتار
`visible`	بیرون می‌زند (پیش‌فرض)
`hidden`	بریده می‌شود
`scroll`	همیشه اسکرول‌بار
`auto`	فقط در صورت نیاز اسکرول‌بار

```
/* بریدن متن طولانی با ... */
.title {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
```

اشتباهات رایج

- `absolute` بدون والد `relative` و تعجب از اینکه المان به گوشه‌ی صفحه می‌رود.
- `sticky` بدون `top` یا با والدی که `overflow: hidden` دارد (sticky کار نمی‌کند).
- استفاده از `position: absolute` برای چیدمان کل صفحه (کار Flexbox و Grid است).
- عدد `z-index` خیلی بزرگ (`9999`) به‌جای مدیریت منظم لایه‌ها.

> **تمرین کوچک:** یک کارت با نشان «جدید» در گوشه‌ی بالایش بساز؛ یک هدر `sticky` بساز و با اسکرول ببین چطور می‌چسبد.

۳.۱۰ Flexbox [ضروری]

مفهوم. Flexbox برای چیدمان **یک‌بعدی** (یک ردیف یا یک ستون) است. المان **والد** (Flex Container) را `display: flex` می‌کنیم و فرزندان مستقیم (Flex Items) خودکار چیده می‌شوند.

```
محور اصلی (Main Axis)  ->  جهت flex-direction
محور متقاطع (Cross Axis) -> عمود بر محور اصلی
```

خاصیت‌های Container

```
.row {
  display: flex;
  flex-direction: row;          /* row | row-reverse | column | column-reverse */
  justify-content: space-between; /* چینش روی محور اصلی */
  align-items: center;          /* چینش روی محور متقاطع */
  flex-wrap: wrap;              /* اجازه‌ی رفتن به خط بعد */
  gap: 16px;                    /* فاصله بین آیتم‌ها */
}
```

خاصیت	مقادیر رایج
`flex-direction`	`row` (پیش‌فرض)، `column`
`justify-content`	`flex-start` ، `center` ، `flex-end` ، `space-between` ، `space-around` ، `space-evenly`
`align-items`	`stretch` (پیش‌فرض)، `baseline` ، `flex-end` ، `flex-start` ، `center`
`flex-wrap`	`nowrap` (پیش‌فرض)، `wrap`
`gap`	فاصله‌ی بین آیتم‌ها (به‌جای margin)

RTL: با `dir="rtl"` جهت `row` از راست به چپ است. یعنی `flex-start` «ابتدای محور» (در RTL سمت راست). نیازی به `row-reverse` برای فارسی نیست؛ جهت خودکار درست می‌شود.

خاصیت‌های Item

```
.item {
  flex-grow: 1;      /* سهم از فضای اضافی */
  flex-shrink: 1;    /* اجازه‌ی کوچک شدن */
  flex-basis: 200px; /* اندازه‌ی پایه قبل از رشد/کوچک شدن */
  align-self: flex-end;
  order: 2;          /* تغییر ترتیب بصری (مراقب دسترس‌پذیری باش) */
}

/* Shorthand: flex: grow shrink basis */
.item { flex: 1; }          /* فضا را مساوی تقسیم کن : 0% 1 1 = */
.item { flex: 0 0 200px; }  /* ثابت ۲۰۰px، نه رشد نه کوچک شدن */
```

مثال ۱: نوار ناوبری

```
<header class="navbar">
  <a class="logo" href="/">لوگو</a>
  <nav>
    <ul class="menu">
      <li><a href="/">خانه</a></li>
      <li><a href="/courses">دوره‌ها</a></li>
      <li><a href="/about">درباره</a></li>
    </ul>
  </nav>
  <button type="button" class="cta">ورود</button>
</header>
```

```
.navbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 24px;
  padding: 12px 24px;
}
.menu {
  display: flex;
  gap: 16px;
  list-style: none;
  margin: 0;
  padding: 0;
}
```

توضیح: `.navbar` سه فرزند دارد (لوگو، منو، دکمه). `space-between` اولی را به ابتدا و آخری را به انتها می‌برد. `.menu` خودش یک Flex Container جدا است (Flexbox تو در تو کاملاً رایج است).

مثال ۲: وسط‌چین کامل (مشهورترین الگو)

```
.center {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}
```

مثال ۳: کارت‌ها در یک ردیف با شکستن خودکار

```
.cards {
  display: flex;
  flex-wrap: wrap;
  gap: 24px;
}
.cards .card {
  flex: 1 1 260px;    /* حداقل حدود ۲۶۰px، بعد رشد کن */
}
```

مثال ۴: «فوتر به پایین» (Sticky Footer)

```
body {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}
main { flex: 1; }    /* فضای باقی‌مانده را بگیر */
```

اشتباهات رایج

- نوشتن `display: flex` روی فرزندان به‌جای والد.
- اشتباه گرفتن `justify-content` (محور اصلی) و `align-items` (محور متقاطع)؛ در `column` جایشان عوض می‌شود.
- `margin` برای فاصله‌ی بین آیتم‌ها به‌جای `gap`.
- انتظار اینکه `flex-wrap` بدون `flex-basis` یا عرض مناسب خوب بشکند.
- استفاده از `order` که ترتیب بصری را از ترتیب خواندن و Tab جدا می‌کند.

> **تمرین کوچک:** فقط با Flexbox: (الف) سه دکمه را در یک ردیف با فاصله‌ی مساوی بچین؛ (ب) یک عنصر را دقیقاً وسط صفحه بگذار؛ (پ) سه کارت بساز که در عرض کم به خط بعد بروند.

۳.۱۱ Grid [ضروری]

مفهوم. Grid برای چیدمان **دوبعدی** (هم ردیف و هم ستون) است. اگر هم سطر و هم ستون برایت مهم است، Grid بهتر است.

```
.layout {
  display: grid;
  grid-template-columns: 1fr 2fr 1fr;    /* سه ستون: نسبت ۱:۲:۱ */
  grid-template-rows: auto 1fr auto;
  gap: 16px;
}
```

واحد `fr`: «سهم از فضای باقی‌مانده».

```
/* ۳ ستون مساوی */
grid-template-columns: repeat(3, 1fr);

/* ستون ثابت + ستون انعطاف‌پذیر */
grid-template-columns: 240px 1fr;

/* ستون با حداقل و حداکثر */
grid-template-columns: minmax(200px, 1fr) 2fr;
```

قرار دادن آیتم‌ها

```
.header { grid-column: 1 / -1; }      /* از خط اول تا آخرین خط: کل عرض */
.hero   { grid-column: span 2; }      /* ۲ ستون را بگیر */
.big    { grid-row: span 2; }
```

Grid Areas: چیدمان با اسم

```
<div class="page">
  <header>سربرگ</header>
  <aside>سایدبار</aside>
  <main>محتوا</main>
  <footer>فوتر</footer>
</div>
```

```
.page {
  display: grid;
  grid-template-columns: 220px 1fr;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
  gap: 16px;
  min-height: 100vh;
}
.page > header { grid-area: header; }
.page > aside  { grid-area: sidebar; }
.page > main   { grid-area: main; }
.page > footer { grid-area: footer; }
```

توضیح: ماتریس متنی در `grid-template-areas` خودش شکل صفحه را نشان می‌دهد. برای تغییر چیدمان موبایل، فقط همین ماتریس را در Media Query عوض می‌کنیم. در RTL ستون اول (sidebar) سمت **راست** قرار می‌گیرد.

شبکه‌ی کارت‌ها: Responsive بدون Media Query

```
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 24px;
}
```

معنی: هر ستون حداقل ۲۴۰px، تا جایی که جا هست ستون اضافه کن و فضای باقی‌مانده را مساوی تقسیم کن. این یکی از پرکاربردترین الگوهای CSS است.

Flexbox یا Grid؟

سؤال	پاسخ
یک ردیف یا یک ستون؟ (محتوا تعیین‌کننده‌ی اندازه)	Flexbox
یک شبکه‌ی دوبعدی؟ (طرح تعیین‌کننده)	Grid
چیدمان کل صفحه	Grid
چیدمان اجزای داخل یک کامپوننت (نوار ابزار، دکمه + آیکن)	Flexbox

هر دو را ترکیب می‌کنیم: Grid برای اسکلت صفحه، Flexbox برای داخل هر بخش.

اشتباهات رایج

- استفاده از Grid برای چیدن چند دکمه در یک ردیف (Flexbox ساده‌تر است).
- فراموش کردن `gap` و استفاده از margin.
- تعریف تعداد ستون‌ها ثابت (`repeat(4, 1fr)`) و شکستن در موبایل؛ از `minmax` / `auto-fit` یا Media Query استفاده کن.
- آیتم‌هایی که با `min-width` بزرگ از ستون بیرون می‌زنند.

> **تمرین کوچک:** با `grid-template-areas` چیدمان «سربرگ، سایدبار، محتوا، فوتر» را بساز. بعد یک شبکه‌ی ۶ کارتی با `auto-fit` بساز و عرض پنجره را تغییر بده.

۳.۱۲ طراحی واکنش‌گرا (Responsive) و Media Query [ضروری]

مفهوم. صفحه باید روی موبایل، تبلت و دسکتاپ خوب دیده شود. سه جزء اصلی:

1. **تگ** `viewport` در HTML (فصل ۲).
2. **چیدمان انعطاف‌پذیر** (Flexbox، Grid، تصویر واکنش‌گرا). `max-width`، `fr`، `%`
3. **Media Query** برای تغییر استایل در اندازه‌های مختلف.

```
/* تصاویر و ویدیو هرگز از والد بیرون نزنند */
img, video {
  max-width: 100%;
  height: auto;
  display: block;
}
```

Media Query

```
.layout { display: grid; gap: 16px; }

@media (min-width: 768px) {
  .layout { grid-template-columns: 240px 1fr; }
}

@media (min-width: 1200px) {
  .container { max-width: 1140px; margin-inline: auto; }
}
```

Mobile-First: ابتدا استایل موبایل (ساده‌ترین حالت) را بنویس و با `min-width` برای صفحه‌های بزرگ‌تر **اضافه** کن.

```
/* پایه: موبایل */
.menu { display: none; }

/* از ۷۶۸px به بالا */
@media (min-width: 768px) {
  .menu { display: flex; }
}
```

رویکرد	شکل	مزیت
Mobile-First	`min-width`	کد ساده‌تر برای موبایل، بار کمتر؛ توصیه‌شده
Desktop-First	`max-width`	برای بازنویسی سایت‌های قدیمی

نقاط شکست (Breakpoints): اعداد ثابت و قطعی نیستند. رایج: ۶۰۰، ۷۶۸، ۱۰۲۴، ۱۲۰۰ پیکسل. بهتر است **جایی که طرح خراب می‌شود** بشکنی، نه بر اساس مدل دستگاه.

انواع دیگر:

```
@media (prefers-color-scheme: dark) { /* تم تیره */ }
@media (prefers-reduced-motion: reduce) { /* کاهش انیمیشن */ }
@media (orientation: landscape) { }
@media print { /* چاپ */ }
```

اشتباهات رایج

- فراموش کردن تگ `viewport` (Media Queryها روی موبایل درست کار نمی‌کنند).
- عرض‌های `px` ثابت برای کانتینر (اسکرول افقی در موبایل).
- ترکیب نادرست `min-width` و `max-width` در یک پروژه (سردرگمی).
- تست فقط با تغییر عرض پنجره؛ از Device Toolbar در DevTools و یک گوشی واقعی هم استفاده کن.

> **تمرین کوچک:** صفحه‌ای بساز که در موبایل تک‌ستون، در تبلت دوستونه و در دسکتاپ سه‌ستونه باشد (یک بار با Media Query و یک بار با `auto-fit`).

۳.۱۳ Transform، Transition و Animation [ضروری]

Transition: تغییر نرم بین دو حالت

```
.btn {
  background: #2563eb;
  color: #fff;
  padding: .75rem 1.5rem;
  border-radius: 8px;
  transition: background-color .2s ease, transform .2s ease;
}
.btn:hover {
  background: #1d4ed8;
  transform: translateY(-2px);
}
```

`transition: property duration timing-function delay`

توضیح: وقتی ماوس روی دکمه می‌رود، رنگ و موقعیت در ۰.۲ ثانیه نرم عوض می‌شود، نه ناگهانی. `transition` را روی **حالت اصلی** (نه فقط `:hover`) بنویس تا هم رفتن و هم برگشتن نرم باشد.

محدودیت: همه‌ی خاصیت‌ها قابل‌انیمیشن نیستند. مثلاً `display` را نمی‌شود transition داد.

Transform: تغییر شکل بدون به‌هم‌ریختن چیدمان

```
.box {
  transform: translate(20px, 10px);   /* جابه‌جایی */
  transform: scale(1.1);              /* بزرگ‌نمایی */
  transform: rotate(45deg);           /* چرخش */
  transform: translateY(-4px) scale(1.05);  /* ترکیب (از چپ به راست اعمال می‌شود) */
}
```

Transform جایگاه المان را در چیدمان تغییر **نمی‌دهد** (فقط ظاهر آن را)؛ برای همین برای انیمیشن مناسب است.

Animation با `@keyframes`

```
@keyframes fade-in-up {
  from { opacity: 0; transform: translateY(16px); }
  to   { opacity: 1; transform: translateY(0); }
}

.card {
  animation: fade-in-up .5s ease-out both;
}

/* اسپینر بارگذاری */
@keyframes spin { to { transform: rotate(360deg); } }
.spinner {
  width: 32px; height: 32px;
  border: 4px solid #e2e8f0;
  border-top-color: #2563eb;
  border-radius: 50%;
  animation: spin .8s linear infinite;
}
```

خاصیت Animation	توضیح
`animation-duration`	مدت
`animation-timing-function`	`ease` ، `linear` ، `ease-in-out`
`animation-iteration-count`	تعداد تکرار یا `infinite`
`animation-fill-mode`	`both` تا حالت اول و آخر حفظ شود

Transition یا Animation؟ Transition برای **واکنش به تغییر وضعیت** (hover، focus، کلاس جدید)؛ Animation برای **حرکتی که خودش شروع می‌شود** یا چند مرحله‌ای.

نکات عملکرد

- ترجیحاً فقط `transform` و `opacity` را انیمیت کن (روان‌ترین‌اند). انیمیشن `width` ، `height` ، `left` ، `top` چیدمان را دوباره محاسبه می‌کند و کند است.
- انیمیشن را کوتاه و ظریف نگه دار (حدود ۱۵۰ تا ۴۰۰ میلی‌ثانیه برای تعاملات).
- کاربرانی که «کاهش حرکت» را روشن کرده‌اند باید احترام ببینند (در بخش پیشرفته: `prefers-reduced-motion`).

اشتباهات رایج

- انیمیشن‌های پرزرق‌وبرق و طولانی.
- انیمیشن `margin` یا `top` به‌جای `transform` .
- فراموش کردن `animation-fill-mode` و پرش به حالت اول در پایان.
- `transition: all` در همه‌جا (هزینه و رفتار غیرمنتظره)؛ خاصیت‌ها را نام ببر.

> **تمرین کوچک:** یک کارت بساز که با hover کمی بالا برود و سایه‌اش بیشتر شود؛ یک اسپینر و یک پیام ورودی با `fade-in-up` بساز.

تمرین‌های CSS متوسط

ساده

1. یک هدر `sticky` با لوگو و منوی Flex بساز.
2. دکمه‌ای با hover و transition نرم بساز.
3. سه کارت هم‌اندازه را در یک ردیف Flex بچین.

متوسط

1. صفحه‌ی «سربرگ، سایدبار، محتوا، فوتر» را با Grid Areas بساز و در موبایل تک‌ستونی کن.
2. گالری ۸ تصویری با `auto-fit` و `object-fit: cover` بساز (تصاویر هم‌اندازه).
3. منوی ناوبری را در موبایل عمودی و در دسکتاپ افقی کن (Mobile-First).

چالشی

1. بدون JavaScript، یک «منوی همبرگری» ساده با تکنیک `input[type=checkbox]:checked + nav` بساز (و محدودیت‌هایش در دسترس‌پذیری را بنویس؛ بعد از فصل DOM با JS درست‌ترش می‌کنیم).

آیا CSS متوسط را بلدم؟

سؤالات مفهومی

1. چرا `position: absolute` بدون والد `relative` به گوشه‌ی صفحه می‌رود؟
2. چه موقع Flexbox و چه موقع Grid؟ دو مثال واقعی بزن.
3. چرا `justify-content` و `align-items` در `flex-direction: column` جابه‌جا می‌شوند؟
4. چرا Mobile-First توصیه می‌شود؟
5. چرا `transform: translateX()` بهتر از انیمیت `left` است؟
6. تفاوت `flex: 1` و `flex: 0 0 200px` چیست؟

تمرین کوتاه (کد را بخوان)

1. چرا فرزند در این کد وسط نیست و چه چیزی باید اصلاح شود؟

```
.parent { display: flex; height: 300px; }
.child { align-items: center; justify-content: center; }
```

1. با این HTML و CSS، هر ستون چقدر عرض می‌گیرد؟

```
.row { display: flex; width: 900px; }
.a { flex: 1; }
.b { flex: 2; }
.c { flex: 0 0 150px; }
```

1. چرا هدر این صفحه با اسکرول نمی‌چسبد؟

```
.page { overflow: hidden; }
.page header { position: sticky; }
```

پروژه‌ی سطح CSS متوسط: «Landing Page و صفحه‌ی محصول ریسپانسیو»

- **هدف:** استایل‌دهی دو صفحه‌ی پروژه‌ی HTML پیشرفته (فصل ۲) به شکل کاملاً واکنش‌گرا.
- **امکانات:**
- Landing Page: هدر `sticky` با منوی Hero، Flex تمام‌عرض با دکمه‌ی CTA، شبکه‌ی ویژگی‌ها (Grid)، نظرات کاربران، FAQ، فرم خبرنامه، فوتر.
- صفحه‌ی محصول: گالری تصاویر (Grid)، ستون اطلاعات، جدول مشخصات، دکمه‌ی افزودن به سبد با hover و transition.
- هر دو: سه حالت چیدمان (موبایل، تبلت، دسکتاپ)، انیمیشن ورود برای بخش‌ها.
- **مهارت‌ها:** Transition، Media Query، Position، Grid، Flexbox و `object-fit`، Animation.
- **محدودیت‌ها:** بدون فریم‌ورک (Bootstrap، Tailwind) و بدون JavaScript؛ Mobile-First؛ بدون `!important`؛ بدون ارتفاع ثابت `px` برای بخش‌های متنی.
- **معیار قبولی:**
- روی عرض‌های ۳۲۰ و ۷۶۸ و ۱۴۴۰ پیکسل اسکرول افقی وجود ندارد.
- تصاویر هیچ‌وقت کشیده یا بیرون‌زده نیستند.
- هر دو در پروژه با دلیل مناسب به‌کار رفته‌اند Flexbox و Grid.
- همه‌ی عناصر تعاملی حالت hover و focus دارند.

- انیمیشن‌ها فقط `transform` و `opacity` را تغییر می‌دهند.

بخش C: CSS پیشرفته

۳.۱۴ متغیرهای CSS (Custom Properties) [ضروری]

مفهوم. متغیر CSS مقداری است که یک بار تعریف و در جاهای زیاد استفاده می‌شود. با `--` شروع می‌شود و با `var()` خوانده می‌شود.

```
:root {
  --color-primary: #2563eb;
  --color-text: #1e293b;
  --radius: 12px;
  --space-4: 1rem;
}

.btn {
  background: var(--color-primary);
  border-radius: var(--radius);
  padding: var(--space-4);
}

/* مقدار پیش‌فرض اگر متغیر تعریف نشده بود */
.title { color: var(--color-title, #111); }
```

تفاوت با متغیر Sass: متغیر CSS در **مرورگر** زنده است؛ می‌توان در `@media`، در سطح المان، یا با JavaScript تغییرش داد.

Scope: تغییر مقدار برای یک بخش

```
.card { --card-bg: #fff; background: var(--card-bg); }
.card.featured { --card-bg: #fef9c3; }   /* همان کد، رنگ دیگر */
```

تم تیره (Dark Mode)

```
 :root {
  --bg: #ffffff;
  --text: #0f172a;
}

@media (prefers-color-scheme: dark) {
  :root {
    --bg: #0b1220;
    --text: #e2e8f0;
  }
}

body { background: var(--bg); color: var(--text); }
```

با JavaScript هم می‌توان با یک کلاس یا `data-theme` روی `html` تم را دستی عوض کرد (فصل DOM).

اشتباهات رایج

- نوشتن متغیر خارج از `:root` و انتظار دسترسی سراسری.
- نام‌گذاری متغیر براساس مقدار (`--blue`) به‌جای نقش (`--color-primary`)؛ وقتی رنگ برند عوض شود نام بی‌معنی می‌شود.
- استفاده از `var()` داخل Media Query condition (در شرط `@media`) کار نمی‌کند).
- متغیرهای بی‌نظم؛ یک **سیستم** بساز: رنگ، فاصله، اندازه‌ی فونت، شعاع، سایه.

> **تمرین کوچک:** رنگ‌ها و فاصله‌های صفحه‌ی خودت را به متغیر تبدیل کن؛ بعد با یک `@media (prefers-color-scheme: dark)` تم تیره بساز.

۳.۱۵ `clamp()` ، `max()` ، `min()` ، `calc()` و تایپوگرافی روان [ضروری]

```
 .sidebar { width: calc(100% - 280px); }              /* ترکیب واحدهای مختلف */
.container { width: min(100% - 2rem, 1100px); }      /* هر کدام کوچک‌تر است */
.box { width: max(300px, 30%); }                     /* هر کدام بزرگ‌تر است */
h1 { font-size: clamp(1.75rem, 1.2rem + 2.5vw, 3rem); }
```

- `calc()` : محاسبه با ترکیب واحدها. دور `+` و `-` باید فاصله باشد.
- `min(a, b)` : کوچک‌ترین. `max(a, b)` : بزرگ‌ترین.
- `clamp(min, ترجیحی, max)` : مقدار «ترجیحی» که نه از min کمتر شود نه از max بیشتر.

تایپوگرافی روان (Fluid Typography): `clamp()` اجازه می‌دهد اندازه‌ی فونت بدون Media Query، نرم با عرض صفحه تغییر کند:

```
h1 { font-size: clamp(1.75rem, 1.2rem + 2.5vw, 3rem); }
p  { font-size: clamp(1rem, .95rem + .25vw, 1.125rem); }
```

نکته‌ی دسترس‌پذیری: اگر اندازه‌ی ترجیحی فقط با `vw` باشد، بزرگ‌نمایی مرورگر روی آن اثر نمی‌گذارد؛ ترکیب `rem + vw` (مثل بالا) این مشکل را کم می‌کند.

> **تمرین کوچک:** کانتینری بساز که همیشه ۱۶px از هر طرف فاصله داشته باشد ولی از ۱۱۰۰px پهن‌تر نشود (با `min()`)، و یک `h1` روان با `clamp()`.

۳.۱۶ Specificity، Cascade و Inheritance [ضروری]

وقتی چند قانون به یک المان اعمال شوند و با هم تعارض داشته باشند، مرورگر برنده را به این ترتیب انتخاب می‌کند:

1. **اهمیت و منشأ (ساده‌شده):** `!important` قوی‌تر از معمولی است، و استایل نویسنده (تو) قوی‌تر از پیش‌فرض مرورگر.
2. **Specificity (خاص‌بودن) Selector**
3. **ترتیب نوشتن:** اگر باز هم مساوی بود، **آخری** برنده است.

محاسبه‌ی Specificity

به‌صورت یک عدد سه‌قسمتی `(ID، Class، Element)`:

نوع	وزن
Inline style	بالاتر از همه‌ی Selectorها
`#id`	(۱، ۰، ۰)
`.class`، `[attr]`، `:pseudo-class`	(۰، ۱، ۰)
`element`، `::pseudo-element`	(۰، ۰، ۱)
`*` و combinatorها	صفر

```
p { color: black; }              /* (0,0,1) */
.note { color: blue; }           /* (0,1,0) */
p.note { color: green; }         /* (0,1,1) */
#intro { color: red; }           /* (1,0,0) */
nav ul li a:hover { color: tomato; }   /* (0,1,4) */
```

مقایسه از چپ به راست است: یک ID از هر تعداد کلاس قوی‌تر است.

!important: بالاترین اولویت (پایین‌ترین راه). به‌جز موارد خاص (مثلاً کلاس‌های ابزاری مثل `.hidden` یا بازنویسی استایل کتابخانه‌ی بیرونی)، استفاده نکن؛ چون دیباگ را سخت و جنگ Specificity ایجاد می‌کند.

قاعده‌ی عملی:

- Selectorها را **کوتاه و کم‌وزن** بنویس (یک کلاس).
- از `id` برای استایل استفاده نکن.
- به‌جای افزایش Specificity، **ساختار** را درست کن یا ترتیب را عوض کن.

Inheritance (وراثت)

بعضی خاصیت‌ها از والد به فرزند **ارث می‌رسند** و بعضی نه:

ارث می‌رسند	ارث نمی‌رسند
`color`، `font-*`، `line-height`، `text-align`، `direction`، `visibility`	`margin`، `padding`، `border`، `background`، `width`، `height`، `display`، `position`

```
body { font-family: "Vazirmatn", sans-serif; color: #1e293b; }  /* همه‌ی فرزندان می‌گیرند */
a { color: inherit; }   /* از والد بگیر */
```

کلیدواژه‌های کنترل: `inherit` (از والد)، `initial` (مقدار پیش‌فرض خود CSS)، `unset` ، `revert` .

`@layer` : کنترل Cascade در پروژه‌های بزرگ [مفید حرفه‌ای]

```
@layer reset, base, components, utilities;

@layer base { a { color: blue; } }
@layer components { .btn { color: white; } }
```

لایه‌ی بعدی (بر اساس ترتیب تعریف) بر قبلی **بدون توجه به Specificity** غلبه می‌کند. برای هماهنگ کردن استایل کتابخانه‌ها و کد خودت مفید است.

دیباگ در DevTools

تب **Styles**: قوانین اعمال‌شده از قوی به ضعیف، قوانین خط‌خورده (override شده)، و تب **Computed** برای مقدار نهایی هر خاصیت و اینکه از کدام قانون آمده.

اشتباهات رایج

- جنگ Specificity: استایل را قوی‌تر می‌کنی (`#app .card .title`) و بعد باید باز هم قوی‌ترش کنی.
- فکر کردن به اینکه ترتیب `class` در `class="a b"`) HTML) مهم است (مهم ترتیب **قوانین در CSS** است).
- فراموش کردن اینکه `margin` و `padding` ارث نمی‌رسند.
- استفاده از `!important` در همه‌جا.

> **تمرین کوچک:** سه قانون با Specificity متفاوت روی یک `p` بنویس و پیش از دیدن نتیجه حدس بزن کدام رنگ می‌شود. بعد در DevTools تأیید کن.

۳.۱۷ Selectorهای پیشرفته [مفید حرفه‌ای]

```
/* :is(): ← چند selector را کوتاه می‌کند */
:is(h1, h2, h3) a { color: inherit; }

/* :where(): ← مثل :is() ولی با Specificity صفر (عالی برای Reset و پایه) */
:where(ul, ol) { padding-inline-start: 1.25rem; }

/* :not() */
button:not(:disabled):hover { background: #1d4ed8; }

/* :has(): ← انتخاب والد بر اساس فرزند */
.card:has(img) { grid-template-columns: 120px 1fr; }
label:has(input:checked) { background: #eff6ff; }

/* :focus-visible و :focus-within */
a:focus-visible { outline: 3px solid #2563eb; outline-offset: 2px; }
.field:focus-within { border-color: #2563eb; }

/* nth-child: با فرمول */
li:nth-child(odd) { }
li:nth-child(3n + 1) { }
li:nth-last-child(-n + 2) { }

/* Attribute Selector پیشرفته */
a[href^="https://"]:not([href*="mysite.com"])::after { content: " ↗"; }
```

Selector	کاربرد
`:is()`	کوتاه‌نویسی، Specificity = بزرگ‌ترین آرگومان
`:where()`	کوتاه‌نویسی، Specificity = **صفر**
`:not()`	استثنا
`:has()`	«والدی که شامل … است»؛ ابزاری که سال‌ها از آن محروم بودیم
`:focus-visible`	فوکوس فقط وقتی لازم است (کیبورد)
`:focus-within`	والدی که فرزندش فوکوس دارد

> پشتیبانی `:has()` و ویژگی‌های جدید امروز در مرورگرهای اصلی خوب است، ولی پیش از استفاده در پروژه‌ی واقعی، جدول پشتیبانی را در caniuse.com یا MDN ببین.

اشتباه رایج: استفاده از `:focus` برای همه‌چیز (حلقه‌ی فوکوس هنگام کلیک ماوس هم نمایش داده می‌شود)؛ `:focus-visible` رفتار بهتر دارد.

۳.۱۸ Stacking Context [مفید حرفه‌ای]

مسئله‌ی کلاسیک: المانی `z-index: 9999` دارد ولی باز هم زیر المان دیگری است. علت: **Stacking Context**.

هر Stacking Context یک «جزیره‌ی مستقل» لایه‌هاست: `z-index` فرزندان فقط **داخل همان جزیره** با هم مقایسه می‌شوند. المان‌هایی که Stacking Context جدید می‌سازند شامل:

- `position` غیر static همراه با `z-index` غیر `auto`
- `opacity` کمتر از ۱
- `transform`، `filter`، `perspective`
- `position: fixed` و `sticky`
- المان Flex/Grid با `z-index`
- `isolation: isolate` (روش **صریح و تمیز** برای ساختن آگاهانه)

```
 .card { position: relative; z-index: 1; }
.card .tooltip { position: absolute; z-index: 9999; }
/* tooltip فقط بالاتر از بقیه‌ی فرزندان .card است، نه از کارت‌های مجاور با z-index بالاتر */
```

راه‌حل: ساختار را بررسی کن: المان شکایت‌کننده کدام والد را دارد که Stacking Context ساخته؟ یک مدیریت منظم `z-index` با متغیرها بساز:

```
 :root {
  --z-dropdown: 100;
  --z-sticky: 200;
  --z-modal: 1000;
  --z-toast: 1100;
}
```

و برای کامپوننت‌ها از `isolation: isolate` استفاده کن تا z-indexهای داخلی بیرون نشت نکنند.

۳.۱۹ Flexbox و Grid پیشرفته [مفید حرفه‌ای]

Flexbox

چرا متن طولانی از Flex Item بیرون می‌زند؟ پیش‌فرض `min-width: auto` است، یعنی آیتم نمی‌تواند از اندازه‌ی محتوایش کوچک‌تر شود.

```
.row { display: flex; gap: 16px; }
.row .content { flex: 1; min-width: 0; }   /* اجازه‌ی کوچک شدن و بریده شدن متن */
```

ترفند `margin: auto` در Flex:

```
.navbar { display: flex; }
.navbar .logout { margin-inline-start: auto; }  /* آخرین آیتم را به انتها می‌فرستد */
```

ترتیب رشد و کوچک شدن: `flex-basis` اندازه‌ی اولیه است؛ بعد `flex-grow` فضای اضافی و `flex-shrink` کمبود فضا را بین آیتم‌ها تقسیم می‌کند. `flex: 1 1 0` یعنی «همه از صفر شروع کنند و مساوی تقسیم شوند».

Grid

```
/* ستون‌های خودکار + ردیف‌هایی که محتوا تعیین می‌کند */
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(180px, 1fr));
  grid-auto-rows: 160px;
  gap: 12px;
}
.gallery .wide { grid-column: span 2; }

/* وسط‌چین سریع */
.hero { display: grid; place-items: center; min-height: 60vh; }

/* نسبت ابعاد */
.thumb { aspect-ratio: 16 / 9; object-fit: cover; width: 100%; }
```

تفاوت	`auto-fit`	`auto-fill`
رفتار با ستون‌های خالی	ستون‌های خالی **جمع می‌شوند** و آیتم‌ها کش می‌آیند	ستون‌های خالی **باقی می‌مانند**

Subgrid (ارث بردن خطوط شبکه از والد، برای هم‌ترازی عناصر داخلی کارت‌ها) در مرورگرهای مدرن پشتیبانی می‌شود. [پیشرفته/اختیاری] پشتیبانی را پیش از استفاده بررسی کن.

اشتباهات رایج

- فراموش کردن `min-width: 0` و سرریز متن.
- `auto-fit` و `auto-fill` را یکسان فرض کردن.
- استفاده از Grid با ارتفاع ثابت و محتوای متغیر.

۳.۲۰ Container Queries [پیشرفته/اختیاری]

Media Query به اندازه‌ی **صفحه** نگاه می‌کند؛ Container Query به اندازه‌ی **والد کامپوننت**. این کار کامپوننت‌ها را واقعاً مستقل می‌کند: یک کارت وقتی در سایدبار باریک است ساده و وقتی در ستون پهن است افقی می‌شود، فارغ از عرض صفحه.

```
 .card-wrapper {
  container-type: inline-size;
  container-name: card;
}

.card { display: grid; gap: 12px; }

@container card (min-width: 480px) {
  .card { grid-template-columns: 160px 1fr; }
}
```

توضیح: `container-type: inline-size` والد را «کانتینر قابل‌پرس‌وجو» می‌کند. `@container` شرط را روی عرض آن والد می‌سنجد. (باید المان داخل کانتینر باشد، نه خود کانتینر.)

> پشتیبانی در مرورگرهای مدرن خوب است؛ پیش از استفاده در محصول واقعی، پشتیبانی هدف پروژه‌ات را بررسی کن.

۳.۲۱ معماری CSS و نام‌گذاری [مفید حرفه‌ای]

مشکل

CSS **سراسری** است؛ هر قانون می‌تواند روی هر جای صفحه اثر بگذارد. در پروژه‌ی بزرگ، بدون نظم، تغییر یک قانون جای دیگری را خراب می‌کند.

BEM: یک قرارداد نام‌گذاری

Block، **E**lement، **M**odifier:

```
<article class="card card--featured">
  <img class="card__image" src="..." alt="..." />
  <h3 class="card__title">عنوان</h3>
  <button class="card__button card__button--primary">خرید</button>
</article>
```

```
.card { }                    /* Block: کامپوننت مستقل */
.card__title { }             /* Element: بخشی از Block (دو underline) */
.card--featured { }          /* Modifier: نسخه یا وضعیتی از Block (دو خط تیره) */
```

مزیت‌ها: همه‌ی Selectorها یک کلاس‌اند (Specificity یکسان و کم)؛ از روی نام می‌فهمی هر چیز به کجا تعلق دارد؛ کامپوننت‌ها بازاستفاده‌پذیرند.

اصول معماری

- **کامپوننت‌محور:** هر بخش UI (دکمه، کارت، نوار ناوبری) یک بلوک مستقل با فایل یا بخش خودش.
- **Selectorهای کم‌عمق:** حداکثر یک یا دو سطح؛ از `.header .nav ul li a` پرهیز کن.
- **بدون وابستگی به ساختار HTML:** `.card__title` بهتر از `.card > div > h3`.
- **کلاس ابزاری** برای موارد کوچک تکراری (`.container`، `.visually-hidden`) با احتیاط.

ساختار فایل‌ها

```
css/
|-- base/
|   |-- reset.css
|   |-- variables.css
|   `-- typography.css
|-- layout/
|   |-- container.css
|   `-- grid.css
|-- components/
|   |-- button.css
|   |-- card.css
|   `-- navbar.css
`-- main.css        <- فایل ورودی، بقیه را import می‌کند
```

CSS Reset / Normalize

مرورگرها استایل پیش‌فرض متفاوتی دارند. یک Reset کوچک مدرن:

```
 *, *::before, *::after { box-sizing: border-box; }

body { margin: 0; line-height: 1.6; -webkit-font-smoothing: antialiased; }

img, picture, video, svg { display: block; max-width: 100%; height: auto; }

input, button, textarea, select { font: inherit; }

p, h1, h2, h3, h4 { overflow-wrap: break-word; }

@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { animation-duration: .01ms !important; transition-
duration: .01ms !important; }
}
```

> **تفاوت Reset و Normalize:** Reset همه‌چیز را صفر می‌کند؛ Normalize تفاوت‌های مرورگرها را هم‌سطح می‌کند و پیش‌فرض‌های مفید را نگه می‌دارد. امروز معمولاً یک «Reset مدرن و کوچک» مثل بالا کافی است.

CSS Nesting (مدرن)

```
.card {
  padding: 1rem;

  & .title { font-size: 1.25rem; }
  &:hover { box-shadow: 0 4px 12px rgb(0 0 0 / .1); }
  @media (min-width: 768px) { padding: 1.5rem; }
}
```

در مرورگرهای مدرن بدون Sass کار می‌کند؛ ولی عمق Nesting را زیاد نکن. (پشتیبانی را بررسی کن.)

> **خارج از این جزوه:** CSS Modules، CSS-in-JS، Tailwind، Sass/SCSS. بعد از React با آن‌ها آشنا می‌شوی؛ اصول بالا در همه‌ی آن‌ها لازم است.

۳.۲۲ دسترس‌پذیری و عملکرد در CSS [ضروری]

حالت Focus

هرگز `outline: none` را بدون جایگزین نگذار؛ کاربر کیبورد گم می‌شود.

```
 :focus-visible {
  outline: 3px solid var(--color-primary);
  outline-offset: 3px;
}
```

`prefers-reduced-motion`

بعضی کاربران (مثلاً با مشکلات تعادل یا تشنج) انیمیشن را در سیستم‌عامل خاموش کرده‌اند:

```
 .card { transition: transform .3s; }

@media (prefers-reduced-motion: reduce) {
  .card { transition: none; }
  * { scroll-behavior: auto !important; }
}
```

سایر نکات دسترس‌پذیری

- **کنتراست** رنگ‌ها (حداقل ۴.۵ به ۱ برای متن معمولی).
- **اندازه‌ی هدف لمسی:** دکمه‌ها و لینک‌ها حداقل حدود ۴۴ در ۴۴ پیکسل در موبایل.
- **رنگ تنها** معنا را منتقل نکند (علامت یا متن هم بگذار).
- **فونت** `rem` تا زوم و تنظیمات کاربر عمل کند.
- **مخفی‌سازی فقط بصری** (متن برای screen reader ولی نامرئی):

```
 .visually-hidden {
  position: absolute;
  width: 1px; height: 1px;
  margin: -1px; padding: 0;
  overflow: hidden;
  clip-path: inset(50%);
  white-space: nowrap;
  border: 0;
}
```

- `display: none` **و** `visibility: hidden` المان را از screen reader هم پنهان می‌کنند (برای حالت مخفی واقعی مناسب‌اند).
- **Skip Link** (ظاهر شدن هنگام فوکوس):

```
 .skip-link {
  position: absolute;
  inset-inline-start: 1rem;
  top: -4rem;
  background: #000; color: #fff; padding: .5rem 1rem;
}
.skip-link:focus { top: 1rem; }
```

ملاحظات عملکرد

- انیمیشن فقط `transform` و `opacity`.
- فونت: تنها وزن‌های لازم، فرمت `woff2`، `font-display: swap`.
- حجم CSS را کم و بی‌استفاده‌ها را حذف کن.
- تصاویر پس‌زمینه‌ی بزرگ را بهینه کن.
- از Selectorهای فوق‌العاده پیچیده‌ی بی‌دلیل پرهیز کن (اثر عملکردی کوچک است، ولی نگهداری را سخت می‌کند).
- در صفحه‌های بسیار طولانی، `content-visibility: auto` می‌تواند رندر بخش‌های خارج از دید را به تعویق بیندازد. [پیشرفته/اختیاری]

> **ارتباط با فصل‌های بعد:** در فصل Performance Basics و Accessibility دوباره به این نکات برمی‌گردیم.

۳.۲۳ ویژگی‌های مدرن CSS: مرور سریع [مفید حرفه‌ای]

ویژگی	کاربرد
`gap`	فاصله در Flex و Grid
`aspect-ratio`	نسبت ابعاد بدون هک `padding-top`
`inset`	کوتاه‌نویسی `top/right/bottom/left`
Logical properties	`inset-inline-start` ، `padding-block` ، `margin-inline` (سازگار با RTL/LTR)
`:has()`	انتخاب والد
Container Queries	کامپوننت واکنش‌گرا به والد
CSS Nesting	تو در تو نوشتن بدون Sass
`@layer`	کنترل Cascade
`clamp()`	اندازه‌ی روان
`color-mix()`	ترکیب رنگ‌ها
`accent-color`	رنگ checkbox و radio بومی
`scroll-snap`	اسکرول مرحله‌ای (کاروسل)
`svh` ، `dvh`	ارتفاع صحیح در موبایل
`text-wrap: balance`	متعادل کردن خطوط عنوان

> **قاعده:** هر ویژگی جدید را قبل از استفاده در پروژه‌ی واقعی با caniuse.com و MDN بسنج. اگر مخاطبانت مرورگرهای قدیمی دارند، یک **fallback** بگذار (قاعده‌ای که اگر ویژگی جدید پشتیبانی نشد، همچنان کار کند).

Logical Properties و RTL (مهم برای فارسی)

```
/* فیزیکی: به جهت وابسته است */
.box { margin-left: 16px; padding-right: 8px; text-align: left; }

/* منطقی: با dir خودکار برمی‌گردد */
.box {
  margin-inline-start: 16px;   /* ابتدای خط */
  padding-inline-end: 8px;     /* انتهای خط */
  text-align: start;
}
```

اگر سایتت دوزبانه (فارسی و انگلیسی) است، ویژگی‌های منطقی مانع نوشتن دو نسخه‌ی CSS می‌شوند.

۳.۲۴ چطور یک UI واقعی و حرفه‌ای را فقط با CSS بسازیم؟ [ضروری]

ظاهر حرفه‌ای از **سیستم** می‌آید، نه از تعداد افکت‌ها. چهار گام:

گام ۱: سیستم طراحی (Design Tokens) با متغیرها

```
:root {
  /* رنگ‌ها */
  --primary-600: #2563eb;
  --primary-700: #1d4ed8;
  --gray-50: #f8fafc;
  --gray-200: #e2e8f0;
  --gray-600: #475569;
  --gray-900: #0f172a;

  /* مقیاس فاصله (مضرب ۴ یا ۸) */
  --space-1: .25rem;
  --space-2: .5rem;
  --space-4: 1rem;
  --space-6: 1.5rem;
  --space-8: 2rem;

  /* مقیاس تایپوگرافی */
  --text-sm: .875rem;
  --text-base: 1rem;
  --text-lg: 1.25rem;
  --text-2xl: 1.75rem;

  /* شعاع و سایه */
  --radius: 12px;
  --shadow-sm: 0 1px 2px rgb(15 23 42 / .08);
  --shadow-md: 0 8px 24px rgb(15 23 42 / .12);
}
```

گام ۲: پایه و Reset (بخش ۳.۲۱).

گام ۳: کامپوننت‌ها با حالت‌ها

هر کامپوننت باید این حالت‌ها را داشته باشد: عادی، disabled، active، focus-visible، hover.

```
 .btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: var(--space-2);
  padding: var(--space-2) var(--space-6);
  border: 0;
  border-radius: var(--radius);
  background: var(--primary-600);
  color: #fff;
  font: inherit;
  font-weight: 600;
  cursor: pointer;
  transition: background-color .2s ease, transform .2s ease;
}
.btn:hover { background: var(--primary-700); transform: translateY(-1px); }
.btn:active { transform: translateY(0); }
.btn:focus-visible { outline: 3px solid var(--primary-600); outline-offset: 3px; }
.btn:disabled { opacity: .5; cursor: not-allowed; transform: none; }

.btn--ghost { background: transparent; color: var(--primary-600); box-shadow: inset
0 0 0 2px var(--primary-600); }
```

گام ۴: چیدمان واکنش‌گرا

```
 .container { width: min(100% - 2rem, 1100px); margin-inline: auto; }

.card-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
  gap: var(--space-6);
}

.card {
  display: flex;
  flex-direction: column;
  gap: var(--space-4);
  padding: var(--space-6);
  background: #fff;
  border: 1px solid var(--gray-200);
  border-radius: var(--radius);
  box-shadow: var(--shadow-sm);
  transition: box-shadow .2s ease, transform .2s ease;
}
.card:hover { box-shadow: var(--shadow-md); transform: translateY(-2px); }
.card__title { font-size: var(--text-lg); color: var(--gray-900); margin: 0; }
.card__text { color: var(--gray-600); margin: 0; }
```

اصول بصری که فرق ایجاد می‌کنند

اصل	چکار کنیم
فاصله‌ی منظم	فقط از مقیاس فاصله استفاده کن؛ فضای خالی کافی بده
سلسله‌مراتب	اندازه، وزن و رنگ را برای تفکیک اهمیت به‌کار ببر
محدودیت رنگ	یک رنگ اصلی، خاکستری‌های خنثی، رنگ‌های وضعیت (موفق/خطا)
همسویی	همه‌چیز روی یک شبکه‌ی نامرئی
سازگاری	شعاع گوشه، سایه و فونت یکسان در همه‌جا
حالت‌ها	hover، focus، disabled، loading
خوانایی	کنتراست، `line-height`، عرض خط (`max-width: 65ch`)

اشتباهات رایج

- مقادیر تصادفی (`margin: 27px` ، `padding: 13px`).
- رنگ‌های زیاد بدون سیستم.
- نبودن حالت focus و disabled.
- سایه‌های سنگین و گوشه‌های خیلی گرد بدون هماهنگی.
- ساختن UI قبل از ساختن قواعد (Tokens).

> **تمرین کوچک:** یک «کارت محصول» کامل (تصویر، عنوان، قیمت، دکمه) با Tokens بساز؛ سپس فقط با تغییر متغیرها یک «تم دوم» بساز.

۳.۲۵ چک‌لیست مهارتی CSS

پایه [ضروری]

- [ ] سه روش افزودن CSS و دلیل ترجیح فایل خارجی را می‌دانم.
- [ ] Selectorهای پایه، ترکیبی، Attribute، Pseudo-class و Pseudo-element را درست می‌نویسم.
- [ ] Box Model و `box-sizing` را توضیح می‌دهم و با DevTools می‌خوانم.
- [ ] واحدها (`px` ، `%` ، `rem` ، `em` ، `vw` ، `vh`) را می‌شناسم و درست انتخاب می‌کنم.
- [ ] `display` و تفاوت روش‌های مخفی‌سازی را می‌دانم.

چیدمان [ضروری]

- [ ] با `position` و `z-index` یک badge، هدر sticky و overlay می‌سازم.
- [ ] با Flexbox ناوبری، وسط‌چین، ردیف کارت و Sticky Footer می‌سازم.
- [ ] با Grid چیدمان صفحه (Areas) و شبکه‌ی کارت واکنش‌گرا می‌سازم.
- [ ] Media Query و Mobile-First را به‌کار می‌برم و روی گوشی واقعی تست می‌کنم.
- [ ] Transition و Animation را با `transform` و `opacity` می‌نویسم.

پیشرفته [ضروری و مفید]

- [ ] متغیرهای CSS، تم تیره و `clamp()` را می‌نویسم.
- [ ] Specificity را حساب و دلیل override شدن را با DevTools پیدا می‌کنم.
- [ ] تفاوت وراثت (چه چیزی ارث می‌رسد) را می‌دانم.
- [ ] Stacking Context و دلیل `z-index` بی‌اثر را می‌فهمم.
- [ ] BEM یا یک قرارداد مشابه را به‌کار می‌برم و ساختار فایل‌ها را منظم می‌کنم.
- [ ] `prefers-reduced-motion`، Focus و کنتراست را در همه‌ی پروژه‌ها رعایت می‌کنم.
- [ ] Logical Properties را برای RTL می‌شناسم.
- [ ] یک UI کامل با Tokens و حالت‌های مختلف می‌سازم.

اختیاری: Nesting، `@layer`، Subgrid، Container Queries؛ مفید است ولی برای ورود به React شرط لازم نیست.

تمرین‌های CSS پیشرفته

ساده

1. رنگ، فاصله و فونت پروژه‌ات را به متغیر ببر.
2. یک `h1` روان با `clamp()` بساز.
3. `:focus-visible` استایل‌دار برای همه‌ی عناصر تعاملی بنویس.

متوسط

1. یک تم تیره با `prefers-color-scheme` اضافه کن.
2. کارت‌های پروژه‌ات را با BEM بازنویسی کن.
3. یک مثال واقعی از Stacking Context بساز که `z-index: 9999` کار نکند و با `isolation` درست کن.

چالشی

1. یک کتابخانه‌ی کوچک کامپوننت (دکمه، کارت، Alert، Input، Badge) با Tokens بساز و در یک صفحه‌ی «Style Guide» نمایش بده.
2. همین کارت را در یک ستون باریک و یک ستون پهن با Container Query به دو شکل نمایش بده.

آیا CSS پیشرفته را بلدم؟

سؤالات مفهومی

1. چرا یک ID در Specificity از ده کلاس قوی‌تر است؟ و چرا این موضوع برای نگهداری مشکل‌ساز است؟
2. چرا `z-index: 9999` گاهی بی‌اثر است؟
3. فرق `:is()` و `:where()` در Specificity چیست و کجا از `:where()` استفاده می‌کنی؟
4. متغیر CSS چه تفاوت اساسی با متغیر Sass دارد؟
5. فرق Media Query و Container Query را با یک مثال بگو.
6. چرا `outline: none` بدون جایگزین مشکل دسترس‌پذیری است؟
7. Logical Properties چه مشکلی را در سایت فارسی و انگلیسی حل می‌کنند؟

تمرین کوتاه (کد را بخوان)

1. این دکمه چه رنگی دارد و چرا؟

```
<button id="save" class="btn primary">ذخیره</button>
```

```
button { color: black; }
.btn { color: blue; }
.btn.primary { color: green; }
#save { color: red; }
button.primary { color: orange; }
```

1. حداقل و حداکثر اندازه‌ی این فونت چقدر است؟

```
h1 { font-size: clamp(1.5rem, 1rem + 3vw, 2.5rem); }
```

1. چرا `tooltip` پشت کارت بعدی می‌رود؟ چطور اصلاحش می‌کنی؟

```
.card { position: relative; z-index: 1; }
.card:nth-child(2) { z-index: 2; }
.tooltip { position: absolute; z-index: 9999; }
```

پروژه‌ی سطح CSS پیشرفته: «Dashboard مدیریتی»

- **هدف:** ساخت یک داشبورد کامل با سیستم طراحی منسجم، تم روشن و تیره و واکنش‌گرا. (JavaScript لازم نیست؛ در فصل‌های بعد همین را زنده می‌کنیم.)
- **امکانات:**
- سایدبار ناوبری (در موبایل جمع‌شونده با تکنیک بدون JS یا فقط افقی)، نوار بالا با جستجو و آواتار.
- کارت‌های آماری (۴ عدد)، یک جدول داده، یک «نمودار» ساده با CSS (مثلاً میله‌ای با `height` درصدی)، لیست فعالیت‌های اخیر.
- حالت‌های دکمه، Badgeهای وضعیت (موفق، هشدار، خطا) و یک Alert.
- تم روشن و تیره با متغیرها و `prefers-color-scheme`.
- **مهارت‌ها:** Flexbox، Grid Areas، متغیرها و Media Query، `clamp()`، Tokens یا Container، BEM، Query، دسترس‌پذیری در CSS.
- **محدودیت‌ها:** بدون فریم‌ورک و بدون JavaScript؛ بدون `!important` (به‌جز `prefers-reduced-motion`)؛ هیچ رنگ یا فاصله‌ی «سخت‌کد» خارج از Tokens؛ Selectorها حداکثر دو سطح.
- **معیار قبولی:**
- تغییر یک متغیر (`--primary-600`) رنگ اصلی کل داشبورد را عوض می‌کند.
- روی ۳۲۰، ۷۶۸ و ۱۴۴۰ پیکسل چیدمان سالم و بدون اسکرول افقی است (به‌جز جدول که در کانتینر خودش اسکرول می‌شود).
- هر دو تم روشن و تیره حداقل کنتراست ۴.۵ به ۱ دارند.
- همه‌ی عناصر تعاملی `:focus-visible` دیدنی دارند.
- با `prefers-reduced-motion: reduce` هیچ انیمیشنی اجرا نمی‌شود.
- هیچ Specificity بالا یا `id` در CSS نیست.

پاسخ‌نامه‌ی مختصر (پس از تلاش خودت ببین)

مقدماتی CSS

- سؤال ۷: عرض واقعی (۲×۵ + ۲×۲۰ + ۲۰۰) ۲۵۰px. با `border-box` همان ۲۰۰px.
- سؤال ۸: `ul li` هر دو `li` (A و B) را می‌گیرد. `ul > li` هم هر دو را می‌گیرد، چون هر کدام فرزند مستقیم یک `ul` است (تفاوت وقتی دیده می‌شود که `li` در عمق بیشتری، مثلاً داخل یک `div` درون `ul` باشد؛ آن‌وقت فقط `ul li` آن را می‌گیرد). `h2 + p` فقط پاراگراف «۱» را می‌گیرد و `h2 ~ p` هر دو پاراگراف را.
- سؤال ۹: `span` المان inline است و `width` و `margin-top` عمودی روی آن اثر ندارند؛ `::before` بدون `content` ساخته نمی‌شود.

CSS متوسط

- سؤال ۷: `align-items` و `justify-content` باید روی **والد** (`.parent`) باشند، نه فرزند.
- سؤال ۸: `.c` = ۱۵۰px ؛ فضای باقی‌مانده ۷۵۰px؛ `.a` = ۲۵۰px و `.b` = ۵۰۰px .
- سؤال ۹: `overflow: hidden` روی والد (و نبود `top`) باعث می‌شود sticky کار نکند.

CSS پیشرفته

- سؤال ۸: قرمز، چون (۱،۰،۰) `#save` از همه قوی‌تر است.
- سؤال ۹: حداقل ۱.۵rem و حداکثر ۲.۵rem (۲۴px و ۴۰px با فونت ۱۶px).
- سؤال ۱۰: هر `.card` با `z-index` یک Stacking Context مستقل می‌سازد؛ `z-index: 9999` تولتیپ فقط داخل همان کارت اعتبار دارد و کارت دوم (با `z-index: 2`) کل کارت اول را همراه تولتیپش زیر خودش می‌گذارد. راه‌حل: برای کارتی که تولتیپش باز است (مثلاً با `:hover` یا `:focus-within`) `z-index` بالاتر بده، یا کارت‌ها را `z-index` یکسان بده تا تولتیپ در یک سطح مقایسه شود.

فصل ۳ تمام شد. با تأیید تو، فصل ۴ (JavaScript: مقدماتی، متوسط، پیشرفته) را شروع می‌کنم.